Exercices — Chapitre 1 : Environnement de développement et premier programme

Cours associé : `C01_Environnement_premier_programme` · **Fichiers de départ :** `02_Exercices/fichiers/E01/`

> **Méthode :** Créez un dossier `E01/` et un fichier par exercice. Compilez toujours avec `g++ -std=c++17 -Wall -Wextra` : un exercice n'est terminé que s'il compile **sans erreur ni avertissement**.

N°	Titre	Niveau	Notions du cours
1.1	Premier programme	1 — Application	§3, §4
1.2	Lire un message d'erreur	2 — Entraînement	§4, §5
1.3	Suivre l'exécution au débogueur	2 — Entraînement	§6
1.4	Trois programmes, trois types d'erreurs	3 — Problème	§2 à §5

Niveau 1 — Application directe

Exercice 1.1 — Premier programme

Objectif : écrire, compiler et exécuter un programme ; constater qu'une modification nécessite une recompilation.

Énoncé :

1. Créez `bonjour.cpp` contenant le programme du cours (§3), puis compilez-le et exécutez-le.
2. Modifiez le message affiché et ajoutez une seconde ligne d'affichage.
3. Exécutez le programme **sans** le recompiler. Qu'observez-vous ? Pourquoi ?
4. Recompilez, puis exécutez à nouveau.

Contraintes : commande de compilation complète ; aucun avertissement.

Résultat attendu : à l'étape 1, le programme affiche :

```
Bonjour !
```

À l'étape 4, il affiche vos deux lignes, chacune sur sa propre ligne.

Niveau 2 — Entraînement

Exercice 1.2 — Lire un message d'erreur

Objectif : localiser une erreur de compilation à partir du message de g++.

Énoncé : le fichier `ex1_2.cpp` contient le code suivant.

```cpp
// Code volontairement incorrect
#include <iostream>

int main() {
    std::cout << "Demarrage" << '\n'
    std::cout << "Systeme pret" << '\n';
    return 0;
}
```

1. Sans compiler, repérez l'erreur.
2. Compilez et relevez, dans le premier message : le fichier, la ligne, la colonne, le type et le texte du message.
3. Le message cite le mot `std`. Sur quelle ligne se trouve ce mot ? Expliquez pourquoi le compilateur a besoin de lire ce mot pour détecter l'oubli.
4. Corrigez le programme, recompilez et exécutez-le.

Résultat attendu : après correction, la compilation ne produit aucun message et le programme affiche :

```
Demarrage
Systeme pret
```

Exercice 1.3 — Suivre l'exécution au débogueur

Objectif : placer un point d'arrêt et exécuter un programme pas à pas.

Énoncé : ouvrez `ex1_3.cpp` dans Visual Studio Code.

```cpp
#include <iostream>

int main() {
    std::cout << "Initialisation" << '\n';
    std::cout << "Lecture de la configuration" << '\n';
    std::cout << "Connexion au reseau" << '\n';
    std::cout << "Pret" << '\n';
    return 0;
}
```

1. Placez un point d'arrêt sur la ligne qui affiche `Initialisation`, puis lancez le débogage (`F5`).
2. Le programme s'arrête. Le message `Initialisation` a-t-il déjà été exécuté ? Justifiez avec le cours (§6.2).
3. Avancez avec `F10` jusqu'à ce que la ligne `return 0;` soit surlignée. Combien de fois avez-vous appuyé sur `F10` ?
4. Terminez l'exécution avec `F5`.

Contraintes : utiliser uniquement les commandes du débogueur ; ne pas modifier le code.

Résultat attendu : les réponses aux questions 2 et 3, et les quatre lignes affichées dans le terminal intégré à la fin de l'exécution.

Niveau 3 — Résolution de problèmes

Exercice 1.4 — Trois programmes, trois types d'erreurs

Objectif : reconnaître une erreur de compilation, un avertissement et une erreur de logique, puis les corriger.

Contexte : un équipement de supervision doit afficher, au démarrage, exactement ces trois lignes, dans cet ordre :

```
Demarrage
Test des capteurs : OK
Systeme pret
```

Un collègue a écrit trois versions du programme : `ex1_4_A.cpp`, `ex1_4_B.cpp` et `ex1_4_C.cpp`, dans les fichiers de départ. Chacune contient **un** problème.

Énoncé :

1. Compilez chaque version avec la commande complète. Pour chacune, notez si un exécutable est produit et recopiez le premier message éventuel.
2. Exécutez les versions qui ont produit un exécutable et comparez leur affichage au cahier des charges.
3. Complétez le tableau :

Version	Exécutable produit ?	Message du compilateur	Type de problème	Correction
A				
B				
C				

4. Corrigez les trois versions et vérifiez que chacune respecte le cahier des charges.
5. Lequel de ces trois problèmes est le plus dangereux dans un vrai projet ? Justifiez.

Contraintes : corriger uniquement la ligne fautive de chaque version ; à la fin, aucune version ne produit de message à la compilation.

Résultat attendu : le tableau complété, trois programmes corrigés qui affichent exactement les trois lignes du cahier des charges, et une réponse argumentée à la question 5.